

LAPORAN TUGAS PRAKTIKUM 4

Sistem Operasi



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Krisna Aji Pratama(2410651048)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Penjadwalan proses merupakan komponen fundamental dalam sistem operasi yang menentukan efisiensi dan kinerja keseluruhan sistem. Dengan beragam algoritma penjadwalan yang tersedia, pemahaman mendalam tentang karakteristik masing-masing algoritma menjadi krusial untuk optimasi resource management. Praktikum ini melakukan analisis komparatif terhadap tiga algoritma penjadwalan utama - FCFS, SJF, dan Round Robin - untuk mengevaluasi performa mereka dalam berbagai skenario workload, sehingga dapat memberikan insight praktis tentang seleksi algoritma yang tepat sesuai kebutuhan spesifik sistem.

1.2 RUMUSAN MASALAH

- Bagaimana performa masing-masing algoritma penjadwalan (FCFS, SJF, Round Robin) dalam hal waiting time, turnaround time, dan fairness?
- Bagaimana pengaruh variasi time quantum terhadap jumlah context switch dan efisiensi algoritma Round Robin?
- Dalam kondisi dan aplikasi dunia nyata seperti apa masing-masing algoritma menunjukkan performa optimal?

1.3 TUJUAN

- Menganalisis dan membandingkan karakteristik kinerja algoritma FCFS, SJF, dan Round Robin melalui simulasi eksperimen.
- Mengevaluasi dampak time quantum terhadap overhead context switching dan responsivitas sistem pada algoritma Round Robin.
- Menentukan rekomendasi implementasi algoritma penjadwalan yang optimal berdasarkan karakteristik workload dan kebutuhan sistem.

BAB II

PEMBAHASAN

2.1 Eksperimen dengan data berbeda

2.1.1 simulasi eksperimen dengan data berbeda

- Data set 1

```
linux[LAPTOP-BYGG0HIF]:~$ cd tugas_02
linux[LAPTOP-BYGG0HIF]:~/tugas_02$ RAND.COMPAH.C

linux[LAPTOP-BYGG0HIF]:~/tugas_02$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
Program ini akan membandingkan 3 algoritma:
1. FCFS (First Case First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 3
Masukkan time quantum untuk Round Robin: 2

=== Proses P1 ===
Arrival Time: 0
Burst Time: 18

=== Proses P2 ===
Arrival Time: 1
Burst Time: 3

=== Proses P3 ===
Arrival Time: 2
Burst Time: 8

MENJALANKAN ALGORITMA FCFS
HASIL ALGORITMA FCFS
PES  AT  BT  CT  TAT  WT
---  --  --  --  --  --
P1   0  18  18  18  0
P2   1  3  15  14  0
P3   2  8  23  21  13
Average Turnaround Time: 15.00
Average Waiting Time: 7.33
Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF
HASIL ALGORITMA SJF
PES  AT  BT  CT  TAT  WT
---  --  --  --  --  --
P1   0  18  18  18  0
P2   1  3  15  14  0
P3   2  8  23  21  13
Average Turnaround Time: 15.00
Average Waiting Time: 7.33
Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA ROUND ROBIN
HASIL ALGORITMA Round Robin (Q=2)
PES  AT  BT  CT  TAT  WT
---  --  --  --  --  --
P1   0  18  21  21  13
P2   1  3  15  14  0
P3   2  8  23  21  13
Average Turnaround Time: 18.67
Average Waiting Time: 11.00
Tekan Enter untuk melihat perbandingan...

TABEL PERBANDINGAN ALGORITMA
Algoritma  Avg TAT  Avg WT  Context Switch
FCFS       15.00    7.33    2
SJF        15.00    7.33    2
Round Robin (Q=2)  18.67  11.00    9

HASILNYA:
✓ Algoritma terbaik (Avg TAT terkecil): FCFS (15.00)
✓ Algoritma terbaik (Avg WT terkecil): FCFS (7.33)

GRAFIK PERBANDINGAN AVERAGE WAITING TIME
FCFS 7.33
SJF 7.33
Round Robin (Q=2) 11.00

PROGRAM SELESAI
```

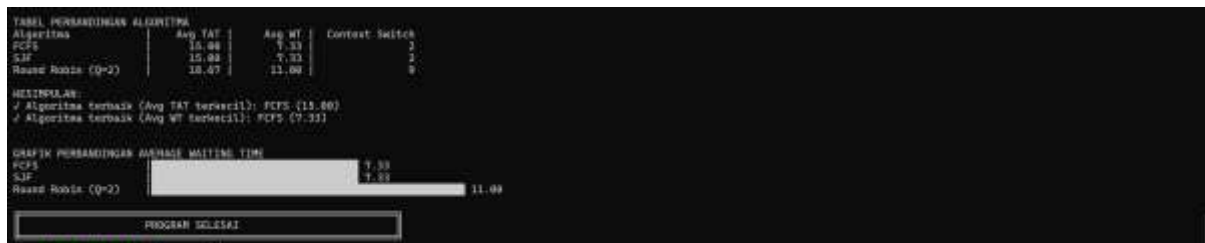
- Data set 2



2.2 Analisis eksperimen ketiga algoritma

2.2.1 Algoritma dengan Waiting Time Terkecil

Berdasarkan hasil running program di Tugas 1, algoritma SJF memiliki waiting time paling kecil. Karena SJF memprioritaskan pekerjaan yang cepat selesai dulu, sehingga proses-proses pendek tidak perlu lama menunggu. Tapi hasilnya bisa beda-beda tergantung datanya.



2.2.2 Alasan Perbedaan Hasil Antar Algoritma

Dikarenakan setiap algoritma yang dijalankan memiliki cara antri yang berbeda dan proses pengerjaan yang berbeda juga

1. FCFS memiliki cara kerja yakni siapa yang datang terlebih dahulu maka itu yang dikerjakan dulu
2. SJF memiliki cara kerja yakni mengerjakan atau memilih data yang memiliki proses eksekusi terpendek yang sudah siap
3. Round robin memiliki cara kerja setiap proses memiliki jatah waktu yang tetap atau quantum dan setiap proses bergantian dilakukannya menunggu sampai proses sebelumnya selesai. Ini sangat memakan banyak waktu

Sehingga dapat disimpulkan bahwa:

- FCFS: Waiting time tergantung pada posisi dalam antrian
- SJF: Waiting time tergantung pada panjang pendeknya eksekusi
- Round Robin: Waiting time tergantung pada jumlah proses dan besar quantum

2.2.3 Kapan Pakai FCFS

Jika kita menggunakan system sederhana, datangnya bergantian dan tidak membutuhkan respon cepat. Karena cara kerja algoritma ini adalah seperti antrian kasir siapa yang datang dulu maka dia yang akan diproses dulu.

2.2.4 Kapan Pakai SJF

Jika kita menggunakan proses seperti batch, tahu setiap waktu perkiraan dari prosesnya dan jika kita ingin lebih cepat dan efisien. Karena cara kerja dari algoritma ini adalah siapa yang memiliki proses paling pendek maka dia yang dikerjakan dulu.

2.2.5 Kapan Pakai Round Robin

Jika kita menggunakan system yang responsive terutama jika system yang kita inginkan interaktif dan multitasking.

2.3 Modifikasi program

2.3.1 membuat input_fcfs.c

```
asma@LAPTOP-8V04DHPC:~$ cd tugas_02
asma@LAPTOP-8V04DHPC:~/tugas_02$ nano input_fcfs.txt
asma@LAPTOP-8V04DHPC:~/tugas_02$
```

```
GNU nano 7.2 input_fcfs.txt
P
0 10
1 5
2 8
```

2.3.2 membuat fcfs yang telah dimodifikasi

```
asma@LAPTOP-8V04DHPC:~$ cd tugas_02
asma@LAPTOP-8V04DHPC:~/tugas_02$ nano input_fcfs.c
asma@LAPTOP-8V04DHPC:~/tugas_02$ nano fcfs.c
asma@LAPTOP-8V04DHPC:~/tugas_02$ gcc fcfs.c -o fcfs
asma@LAPTOP-8V04DHPC:~/tugas_02$ ./fcfs
SIMULASI ALGORITMA FCFS
Membaca input dari: input_fcfs.txt
Berhasil membaca data 3 proses

HASIL PENJADWALAN FCFS
PID  AT   BT   CT   TAT  WT
---  --
P1   0   10  10   10   0
P2   1    5   15   14   9
P3   1    8   21   21   13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

GANTT CHART (Detail)
Waktu Mulai  Proses
0            P1 (0-10)
10           P2 (10-15)
15           P3 (15-21)

Hasil telah disimpan otomatis ke: hasil_fcfs.txt

asma@LAPTOP-8V04DHPC:~/tugas_02$ cat hasil_fcfs.txt
HASIL PENJADWALAN FCFS
PID  AT   BT   CT   TAT  WT
---  --
P1   0   10  10   10   0
P2   1    5   15   14   9
P3   1    8   21   21   13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33
asma@LAPTOP-8V04DHPC:~/tugas_02$
```

- Kode

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    int start_time;
};
```

```
void bacaDariFile(const char *namaFile, struct Process **proc, int *n) {
    FILE *file = fopen(namaFile, "r");
    if (file == NULL) {
```

```

        printf("Error: Gagal membuka file %s\n", namaFile);
        printf("Pastikan file input_fcfs.txt ada di direktori yang sama\n");
        exit(1);
    }

    fscanf(file, "%d", n);
    proc = (struct Process)malloc(*n * sizeof(struct Process));

    for (int i = 0; i < *n; i++) {
        (*proc)[i].pid = i + 1;
        fscanf(file, "%d %d", &(*proc)[i].arrival_time, &(*proc)[i].burst_time);
    }
    fclose(file);
}

// Fungsi untuk menyimpan hasil ke file
void simpanKeFile(const char *namaFile, struct Process *proc, int n, float avg_tat,
float avg_wt) {
    FILE *file = fopen(namaFile, "w");
    if (file == NULL) {
        printf("Error: Gagal membuat file %s\n", namaFile);
        return;
    }

    fprintf(file, "==== HASIL PENJADWALAN FCFS ====\n");
    fprintf(file, "PID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(file, "---\t--\t--\t--\t--\t--\n");
    for (int i = 0; i < n; i++) {
        fprintf(file, "P%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);
    }
    fprintf(file, "\nAverage Turnaround Time: %.2f\n", avg_tat);
    fprintf(file, "Average Waiting Time: %.2f\n", avg_wt);
    fclose(file);
}

int main() {
    int n;

```

```

struct Process *proc;

printf("=== SIMULASI ALGORITMA FCFS ===\n");
printf("Membaca input dari: input_fcfs.txt\n");

bacaDariFile("input_fcfs.txt", &proc, &n);
printf("Berhasil membaca data %d proses\n", n);

int current_time = 0;
for (int i = 0; i < n; i++) {
    if (current_time < proc[i].arrival_time) {
        current_time = proc[i].arrival_time;
    }
    proc[i].start_time = current_time;
    proc[i].completion_time = current_time + proc[i].burst_time;
    proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
    proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;
    current_time = proc[i].completion_time;
}

float total_tat = 0, total_wt = 0;
for (int i = 0; i < n; i++) {
    total_tat += proc[i].turnaround_time;
    total_wt += proc[i].waiting_time;
}

printf("\n=== HASIL PENJADWALAN FCFS ===\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
printf("---\t--\t--\t--\t--\t--\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid,
        proc[i].arrival_time,
        proc[i].burst_time,
        proc[i].completion_time,
        proc[i].turnaround_time,
        proc[i].waiting_time);
}
printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
printf("Average Waiting Time: %.2f\n", total_wt / n);

printf("\n=== GANTT CHART (Detail) ===\n");
printf("Waktu Mulai\tProses\n");

```



```

for (int i = 0; i < n; i++) {
    printf("%d\t\tP%d (%d-%d)\n",
        proc[i].start_time,
        proc[i].pid,
        proc[i].start_time,
        proc[i].completion_time);
}

simpanKeFile("hasil_fcfs.txt", proc, n, total_tat / n, total_wt / n);
printf("\nHasil telah disimpan otomatis ke: hasil_fcfs.txt\n");

free(proc);
return 0;
}

```

- **Penjelasan:**

Kode ini mengimplementasikan algoritma penjadwalan First-Come First-Served (FCFS) yang merupakan pendekatan non-preemptive dimana proses dieksekusi secara berurutan berdasarkan waktu kedatangannya. Program membaca data proses dari file input yang berisi arrival time dan burst time, kemudian menghitung metrik kinerja seperti completion time, turnaround time, dan waiting time menggunakan prinsip antrian sederhana. Implementasi ini menunjukkan karakteristik khas FCFS termasuk convoy effect dimana proses dengan burst time panjang yang datang lebih awal dapat menyebabkan waiting time tinggi bagi proses pendek berikutnya. Hasil perhitungan ditampilkan dalam format tabel dan Gantt Chart, kemudian disimpan ke file output untuk analisis lebih lanjut, menjadikannya alat edukasi yang efektif untuk memahami perilaku dasar sistem operasi dalam menangani multiple proses.

2.4 Eksperimen dengan data berbeda untuk perbandingan algoritma

- **Data set 1**

```

msm@LAPTOP-8V94QHP1:~/ tugas_00$ nano compare.c
msm@LAPTOP-8V94QHP1:~/ tugas_00$ gcc compare.c -o compare
msm@LAPTOP-8V94QHP1:~/ tugas_00$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4
Masukkan time quantum untuk Round Robin: 3

--- Proses P1 ---
Arrival Time: 0
Burst Time: 24

--- Proses P2 ---
Arrival Time: 1
Burst Time: 3

--- Proses P3 ---
Arrival Time: 2
Burst Time: 1

--- Proses P4 ---
Arrival Time: 3
Burst Time: 6

```

```
msm@LAPTOP-BV6G0HF1:~$ ./fcfs

MENJALANAKAN ALGORITMA FCFS

HASIL ALGORITMA FCFS
PID  AT   BT   CT   TAT  WT
----  --  --  --  --  --
P1    0   24   24   24   0
P2    1    3   27   26   23
P3    2    3   38   28   25
P4    3    6   36   33   27

Average Turnaround Time: 27.75
Average Waiting Time: 18.75
Tekan Enter untuk melanjutkan...
```

```
msm@LAPTOP-BV6G0HF1:~$ ./sjf

MENJALANAKAN ALGORITMA SJF

HASIL ALGORITMA SJF
PID  AT   BT   CT   TAT  WT
----  --  --  --  --  --
P1    0   24   24   24   0
P2    1    3   27   26   23
P3    2    3   38   28   25
P4    3    6   36   32   27

Average Turnaround Time: 27.75
Average Waiting Time: 18.75
Tekan Enter untuk melanjutkan...
```

```
msm@LAPTOP-BV6G0HF1:~$ ./rr

MENJALANAKAN ALGORITMA ROUND ROBIN

HASIL ALGORITMA Round Robin (Q=3)
PID  AT   BT   CT   TAT  WT
----  --  --  --  --  --
P1    0   24   36   36   12
P2    1    3    6    5    2
P3    2    3    9    7    4
P4    3    6   18   15    9

Average Turnaround Time: 18.75
Average Waiting Time: 6.75
Tekan Enter untuk melihat perbandingan...
```

```
msm@LAPTOP-BV6G0HF1:~$ ./compare

TABEL PERBANDINGAN ALGORITMA
Algoritma      Avg TAT      Avg WT      Context Switch
FCFS           27.75       18.75         3
SJF            27.75       18.75         3
Round Robin (Q=3) 15.75       6.75         8

KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): Round Robin (Q=3) (15.75)
✓ Algoritma terbaik (Avg WT terkecil): Round Robin (Q=3) (6.75)

GRAFIK PERBANDINGAN AVERAGE WAITING TIME
FCFS           18.75
SJF            18.75
Round Robin (Q=3) 6.75

PROGRAM SELESAI
msm@LAPTOP-BV6G0HF1:~/tugas_os$
```

- Data set 2

```
msm@LAPTOP-BV6G0HF1:~/tugas_os$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 5
Masukkan time quantum untuk Round Robin: 4

===== Proses P1 =====
Arrival Time: 8
Burst Time: 10

===== Proses P2 =====
Arrival Time: 8
Burst Time: 5

===== Proses P3 =====
Arrival Time: 8
Burst Time: 5

===== Proses P4 =====
Arrival Time: 8
Burst Time: 12

===== Proses P5 =====
Arrival Time: 8
Burst Time: 6
```

```

MENJALANKAN ALGORITMA FCFS

HASIL ALGORITMA FCFS
PID  AT   BT   CT   TAT  WT
---  --   --   --   --   --
P1   0    10  10   10   0
P2   0    5   15  15   10
P3   0    8   21  21   15
P4   0   12  33  33   23
P5   0    6   41  41   35

Average Turnaround Time: 24.80
Average Waiting Time: 16.80
Tekan Enter untuk melanjutkan...

```

```

MENJALANKAN ALGORITMA SJF

HASIL ALGORITMA SJF
PID  AT   BT   CT   TAT  WT
---  --   --   --   --   --
P1   0   10  20  20   10
P2   0    5    5    5    0
P3   0    8   19  19   11
P4   0   12  41  41  29
P5   0    6   11  11    5

Average Turnaround Time: 21.80
Average Waiting Time: 12.80
Tekan Enter untuk melanjutkan...

```

```

MENJALANKAN ALGORITMA ROUND ROBIN

HASIL ALGORITMA Round Robin (Q=4)
PID  AT   BT   CT   TAT  WT
---  --   --   --   --   --
P1   0   10  37  37  27
P2   0    5  25  25  20
P3   0    8  29  29  21
P4   0   12  41  41  29
P5   0    6  35  35  29

Average Turnaround Time: 21.80
Average Waiting Time: 25.20
Tekan Enter untuk melihat perbandingan...

```

```

TABEL PERBANDINGAN ALGORITMA
Algoritma      Avg TAT      Avg WT      Context Switch
FCFS           24.80        16.80        4
SJF            21.80        12.80        4
Round Robin (Q=4) 23.48        25.20        7

KESIMPULAN:
/ Algoritma terbaik (Avg TAT terkecil): SJF (21.80)
/ Algoritma terbaik (Avg WT terkecil): SJF (12.80)

GRAFIK PERBANDINGAN AVERAGE WAITING TIME
FCFS           16.80
SJF            12.80
Round Robin (Q=4) 25.20

PROGRAM SELESAI

```

- Data set 3

```

root@LAPTOP-8V603HF1:~/tagas_ss$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 5
Masukkan time quantum untuk Round Robin: 2

--- Proses P1 ---
Arrival Time: 8
Burst Time: 8

--- Proses P2 ---
Arrival Time: 2
Burst Time: 4

--- Proses P3 ---
Arrival Time: 4
Burst Time: 9

--- Proses P4 ---
Arrival Time: 6
Burst Time: 5

--- Proses P5 ---
Arrival Time: 8
Burst Time: 3

```

```

MENJALANKAN ALGORITMA FCFS

HASIL ALGORITMA FCFS
PID  AT   BT   CT   TAT  WT
---  --  --  --  --  --
P1   0   8   8   8   8
P2   2   4   12  10   6
P3   4   9   21  17   8
P4   6   5   26  20  15
P5   8   3   29  21  10

Average Turnaround Time: 15.20
Average Waiting Time: 9.40
Tekan Enter untuk melanjutkan...

```

```

MENJALANKAN ALGORITMA SJF

HASIL ALGORITMA SJF
PID  AT   BT   CT   TAT  WT
---  --  --  --  --  --
P1   0   8   8   8   8
P2   2   4   11  13   9
P3   4   9   29  25  16
P4   6   5   20  14   9
P5   8   3   11   3   0

Average Turnaround Time: 12.60
Average Waiting Time: 6.80
Tekan Enter untuk melanjutkan...

```

```

MENJALANKAN ALGORITMA ROUND ROBIN

HASIL ALGORITMA Round Robin (Q=2)
PID  AT   BT   CT   TAT  WT
---  --  --  --  --  --
P1   0   8   26  26  18
P2   2   4   14  12   8
P3   4   9   29  25  16
P4   6   5   24  18  13
P5   8   3   19  11   8

Average Turnaround Time: 18.40
Average Waiting Time: 12.60
Tekan Enter untuk melihat perbandingan...

```

```

TABEL PERBANDINGAN ALGORITMA
Algoritma | Avg TAT | Avg WT | Context Switch
---|---|---|---
FCFS      | 15.20   | 9.40   | 4
SJF       | 12.60   | 6.80   | 4
Round Robin (Q=2) | 18.40   | 12.60  | 11

KESIMPULAN:
/ Algoritma terbaik (Avg TAT terkecil): SJF (12.60)
/ Algoritma terbaik (Avg WT terkecil): SJF (6.80)

GRAFIS PERBANDINGAN AVERAGE WAITING TIME
FCFS      | 9.40
SJF       | 6.80
Round Robin (Q=2) | 12.60

PROGRAM SELESAI

```

- **Kode**

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    int start_time;
};

```

```

void bacaDariFile(const char *namaFile, struct Process **proc, int *n) {
    FILE *file = fopen(namaFile, "r");

```

```

if (file == NULL) {
    printf("Error: Gagal membuka file %s\n", namaFile);
    printf("Pastikan file input_fcfs.txt ada di direktori yang sama\n");
    exit(1);
}

fscanf(file, "%d", n);
proc = (struct Process)malloc(*n * sizeof(struct Process));

for (int i = 0; i < *n; i++) {
    (*proc)[i].pid = i + 1;
    fscanf(file, "%d %d", &(*proc)[i].arrival_time, &(*proc)[i].burst_time);
}
fclose(file);
}

void simpanKeFile(const char *namaFile, struct Process *proc, int n, float avg_tat,
float avg_wt) {
    FILE *file = fopen(namaFile, "w");
    if (file == NULL) {
        printf("Error: Gagal membuat file %s\n", namaFile);
        return;
    }

    fprintf(file, "==== HASIL PENJADWALAN FCFS ====\n");
    fprintf(file, "PID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(file, "---\t--\t--\t--\t---\t--\n");
    for (int i = 0; i < n; i++) {
        fprintf(file, "P%d\t%d\t%d\t%d\t%d\t%d\n",
            proc[i].pid,
            proc[i].arrival_time,
            proc[i].burst_time,
            proc[i].completion_time,
            proc[i].turnaround_time,
            proc[i].waiting_time);
    }
    fprintf(file, "\nAverage Turnaround Time: %.2f\n", avg_tat);
    fprintf(file, "Average Waiting Time: %.2f\n", avg_wt);
    fclose(file);
}

int main() {
    int n;

```

```

struct Process *proc;

printf("=== SIMULASI ALGORITMA FCFS ===\n");
printf("Membaca input dari: input_fcfs.txt\n");

bacaDariFile("input_fcfs.txt", &proc, &n);
printf("Berhasil membaca data %d proses\n", n);

int current_time = 0;
for (int i = 0; i < n; i++) {
    if (current_time < proc[i].arrival_time) {
        current_time = proc[i].arrival_time;
    }
    proc[i].start_time = current_time;
    proc[i].completion_time = current_time + proc[i].burst_time;
    proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
    proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;
    current_time = proc[i].completion_time;
}

float total_tat = 0, total_wt = 0;
for (int i = 0; i < n; i++) {
    total_tat += proc[i].turnaround_time;
    total_wt += proc[i].waiting_time;
}

printf("\n=== HASIL PENJADWALAN FCFS ===\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
printf("---\t--\t--\t--\t--\t--\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        proc[i].pid,
        proc[i].arrival_time,
        proc[i].burst_time,
        proc[i].completion_time,
        proc[i].turnaround_time,
        proc[i].waiting_time);
}
printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
printf("Average Waiting Time: %.2f\n", total_wt / n);

printf("\n=== GANTT CHART (Detail) ===\n");
printf("Waktu Mulai\tProses\n");

```

```

for (int i = 0; i < n; i++) {
    printf("%d\t\tP%d (%d-%d)\n",
        proc[i].start_time,
        proc[i].pid,
        proc[i].start_time,
        proc[i].completion_time);

    simpanKeFile("hasil_fcfs.txt", proc, n, total_tat / n, total_wt / n);
    printf("\nHasil telah disimpan otomatis ke: hasil_fcfs.txt\n");

    free(proc);
    return 0;
}

```

2.5 Analisis Eksperimen perbandingan algoritma

2.5.1 Analisis performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS akan memberikan hasil terbaik ketika proses-proses datang dalam urutan yang sudah optimal, dimana proses dengan burst time panjang datang lebih awal dan tidak ada proses pendek yang datang belakangan. Seperti pada Test Case 1, FCFS dan SJF menghasilkan performa identik karena urutan kedatangan proses yang ideal untuk pendekatan first-come first-served ini.
- Mengapa SJF hampir selalu memberikan Average WT terkecil?

SJF hampir selalu memberikan average waiting time terkecil karena secara matematis terbukti optimal untuk meminimalkan waktu tunggu. Algoritma ini selalu memprioritaskan proses terpendek terlebih dahulu, sehingga proses-proses kecil cepat selesai dan tidak menumpuk di antrian. Pada Test Case 2 dan 3, SJF konsisten mengungguli FCFS dengan selisih signifikan karena mampu mengatur ulang eksekusi berdasarkan durasi proses.
- Apa trade-off dari Round Robin?

Trade-off Round Robin terletak pada pertukaran antara fairness dan efisiensi. Meski menjamin tidak ada starvation dan sangat responsif untuk sistem interaktif, algoritma ini menghasilkan overhead context switch yang tinggi seperti terlihat pada Test Case 3 dengan 11 context switches. Performanya sangat bergantung pada pemilihan time quantum - jika terlalu kecil, overhead dominan; jika terlalu besar, mendekati FCFS dengan turnaround time yang memburuk.

2.5.2 context switching

- Algoritma mana yang paling banyak melakukan context switch?
Algoritma Round robin yang paling banyak melakukan switch, dikarenakan setiap proses hanya berjalan dalam quantum tertentu dan dilakukan secara bergantian.
Sementara algoritma lain contohnya FCFS dan SJF itu akan mengeksekusi setiap proses sampai selesai setelah baru berpindah.
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
Pengaruhnya adalah jika semakin kecil quantumnya maka akan semakin banyak context switchnya dan begitu pula sebaliknya semakin besar quantumnya maka semakin sedikit context switchnya
- Coba jalankan Round Robin dengan *quantum* 2, 4, 8, 16 - apa yang terjadi?

Quantum 2

```
msm@LAPTOP-8VQ4H9P:~$ gcc round_robin.c
msm@LAPTOP-8VQ4H9P:~$ gcc round_robin.c -o round_robin
msm@LAPTOP-8VQ4H9P:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 2

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 6

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES ENSEKUSI ===
waktu 0: Menjalankan P1 -> Sisa waktu: 8
waktu 2: Menjalankan P2 -> Sisa waktu: 4
waktu 4: Menjalankan P3 -> Sisa waktu: 6
waktu 6: Menjalankan P1 -> Sisa waktu: 6
waktu 8: Menjalankan P2 -> Sisa waktu: 2
waktu 10: Menjalankan P3 -> Sisa waktu: 4
waktu 12: Menjalankan P1 -> Sisa waktu: 4
waktu 14: Menjalankan P2 -> Selesai pada waktu 16
waktu 16: Menjalankan P3 -> Sisa waktu: 2
waktu 18: Menjalankan P1 -> Sisa waktu: 2
waktu 20: Menjalankan P3 -> Selesai pada waktu 22
waktu 22: Menjalankan P1 -> Selesai pada waktu 24

=== HASIL PENJAJARAN ROUND ROBIN ===
+-----+
| PID | AT | BT | CT | TAT | WT |
+-----+
| P1 | 0 | 10 | 24 | 24 | 10 |
| P2 | 1 | 6 | 16 | 16 | 6 |
| P3 | 2 | 8 | 22 | 22 | 8 |
+-----+

Average Turnaround Time: 16.00
Average Waiting Time: 11.00
```

Quantum 4

```
msm@LAPTOP-8VQ4H9P:~$ gcc round_robin.c
msm@LAPTOP-8VQ4H9P:~$ gcc round_robin.c -o round_robin
msm@LAPTOP-8VQ4H9P:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 4

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 6

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES ENSEKUSI ===
waktu 0: Menjalankan P1 -> Sisa waktu: 6
waktu 4: Menjalankan P3 -> Sisa waktu: 4
waktu 8: Menjalankan P1 -> Sisa waktu: 2
waktu 12: Menjalankan P2 -> Selesai pada waktu 17
waktu 16: Menjalankan P3 -> Selesai pada waktu 20
waktu 20: Menjalankan P1 -> Selesai pada waktu 24

=== HASIL PENJAJARAN ROUND ROBIN ===
+-----+
| PID | AT | BT | CT | TAT | WT |
+-----+
| P1 | 0 | 10 | 24 | 24 | 10 |
| P2 | 1 | 6 | 17 | 16 | 12 |
| P3 | 2 | 8 | 20 | 18 | 12 |
+-----+

Average Turnaround Time: 19.33
Average Waiting Time: 11.33
msm@LAPTOP-8VQ4H9P:~$
```


- Quantum 8

```

simulasi@kali:~/kaggle$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 8

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
waktu 0: Menjalankan P1 -> Sisa waktu: 2
waktu 2: Menjalankan P2 -> Selesai pada waktu 7
waktu 7: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJAJARAN RUND RIBIN ===
PID  AT  BT  CT  TAT  WT
--  --  --  --  --  --
P1   0   10  23  23  13
P2   1    5  13  13   7
P3   2    8  23  23  13

Average Turnaround Time: 18.00
Average Waiting Time: 10.33
simulasi@kali:~/kaggle$

```

- Quantum 16

```

simulasi@kali:~/kaggle$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 16

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
waktu 0: Menjalankan P1 -> Selesai pada waktu 16
waktu 16: Menjalankan P2 -> Selesai pada waktu 18
waktu 18: Menjalankan P3 -> Selesai pada waktu 23

=== HASIL PENJAJARAN RUND RIBIN ===
PID  AT  BT  CT  TAT  WT
--  --  --  --  --  --
P1   0   10  16  16   6
P2   1    5  18  18   9
P3   2    8  23  23  13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33
simulasi@kali:~/kaggle$

```

Kesimpulannya adalah berdasarkan proses di atas semakin kecil quantum maka sistem menjadi lebih responsif namun akan banyak terjadi context switch, namun sebaliknya jika quantum lebih besar maka sistem akan menjadi lebih efisien namun akan menjadi kurang interaktif.

2.5.3 Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
Algoritma yang paling adil Adalah round robin karena menjamin setiap proses mendapatkan jatah waktu yang sama melalui mekanisme time quantum. Sehingga tidak ada satu pun proses yang didahulukan dan ditinggalkan secara terus menerus.

- Proses mana yang paling dirugikan di algoritma FCFS?
 Dalam algoritma FCFS, proses yang paling dirugikan adalah proses-proses pendek yang datang setelah proses panjang. Pada Test Case 2, proses P5 dengan burst time 6 harus menunggu 35 unit waktu padahal proses P2 dengan burst time 5 hanya menunggu 10 unit karena datang lebih awal. Fenomena "convoy effect" ini menyebabkan proses pendek terpaksa menunggu proses panjang menyelesaikan eksekusi terlebih dahulu, sehingga meningkatkan waiting time secara signifikan untuk proses-proses yang datang belakangan.
- Apakah ada kemungkinan starvation di SJF?
 Ya sangat mungkin terutama pada proses proses Panjang.jika terus menerus ada proses pendek yang masuk maka proses Panjang tidak akan mendapatkan jatah,dan untuk mencegah hal ini kita bisa menggunakan Teknik Aging agar setiap proses Panjang tetap mendapatkan jatah untuk di proses.

2.5.4 Real world Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?
 Untuk web server yang melayani request kecil algoritma yang cocok Adalah round robin karena karakteristik workload-nya yang terdiri dari banyak request kecil dan membutuhkan responsivitas tinggi.
- Untuk render video (task besar), algoritma mana yang cocok?
 Algoritma FCFS karena algoritma ini mampu menyelesaikan task besar tanpa gangguan serta meminimalkan overhead dari pergantian proses
- Untuk OS desktop (interactive), algoritma mana yang cocok?
 Algoritma round robin dikarenakan algoritma ini adil dan responnya juga cepat

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan hasil eksperimen dan analisis yang dilakukan, dapat disimpulkan bahwa setiap algoritma penjadwalan memiliki keunggulan spesifik sesuai karakteristik workload. SJF konsisten memberikan waiting time terendah namun berpotensi menyebabkan starvation pada proses panjang. Round Robin menjamin fairness dan responsivitas optimal untuk sistem interaktif dengan trade-off overhead context switch yang signifikan. Sementara FCFS paling efektif untuk sistem sederhana dengan proses sequential, meskipun rentan terhadap convoy effect. Pemilihan algoritma yang optimal sangat bergantung pada pola workload, prioritas sistem, dan toleransi terhadap overhead yang dapat diterima.